WADDLE

Project Report

Hindraj Mali
Palak Singhi

Eklavya 2026

Acknowledgements

We would like to sincerely thank our mentors and guides, Gargi Gupta and Abhijeet Bhalerao for their constant guidance, valuable insights, and encouragement throughout the development of this project. Their support and feedback played a significant role in helping us overcome challenges and bring our ideas to reality.

We would also like to extend our gratitude to the **Society of Robotics and Automation (SRA), VJTI**, for providing us with the necessary platform, resources and opportunities to explore and implement our ideas. The exposure and support provided by SRA greatly contributed to our learning and the successful realization of our vision with **"WADDLE"**.

We are truly grateful to everyone who supported us directly or indirectly throughout this journey.

1 Overview

The project "Waddle" is a Reinforcement Learning-based robotic framework developed to enable robots to learn intelligent locomotion and control through interaction with their environment. Rather than relying entirely on predefined motion patterns, Waddle allows a robot to discover effective movement strategies by continuously observing its state, selecting actions, and receiving rewards based on how well it performs a given task.

At the heart of Waddle is the challenge of teaching a robot to achieve stable, responsive, and goal-directed motion. Carefully designed reward functions guide the learning process by encouraging behaviors such as accurate linear velocity tracking, correct angular velocity tracking, forward movement, and overall motion stability. Various Reinforcement Learning algorithms, including Q-Learning, DQN, DDQN, PPO, TRPO, SAC and FlashSAC, are explored to study their effectiveness in learning robust control policies. The framework also provides scope for investigating more efficient off-policy approaches and improving training performance for complex robotic tasks.

Ultimately, Waddle represents a step toward autonomous robots capable of learning how to move, adapt, and perform tasks with minimal human intervention, making robotic control more intelligent, flexible, and scalable.

2 Reinforcement Learning Fundamentals

2.1 Reinforcement Learning Overview

Reinforcement Learning (RL) is a machine learning technique where an agent learns by interacting with an environment. The agent takes actions, receives rewards or penalties, and learns from these experiences. The main goal is to learn the best possible behavior that maximizes the total reward over time. Unlike supervised learning, RL does not require a predefined correct answer for every action. In the Waddle project, RL is used to train the robot to learn walking through repeated interaction with a simulated environment.

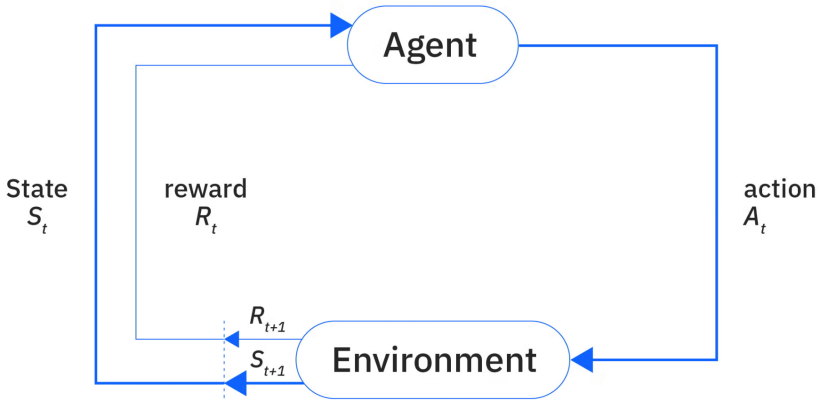

Figure 1: Reinforcement Learning Overview

2.1.1 Agent

The agent is the decision-making part of the system. It observes the current situation and decides what action should be taken.

2.1.2 Environment

The environment is the world in which the agent operates. It receives the action from the agent, applies it, and provides the next observation and reward.

2.1.3 State

The state describes the current condition of the robot. It can contain information such as joint positions, joint velocities, robot orientation, body velocity, and other required measurements. The agent uses this information to decide its next action.

2.1.4 Action

An action is the command given by the agent to the environment. In a robotic system, actions can control joint positions, velocities, or torques.

2.1.5 Reward

A reward is a numerical value given to the agent after it performs an action.

2.2 Policy

A policy defines how the agent chooses an action based on the current observation.

2.3 Value Function

The value function estimates how good the current state is based on the expected future rewards.

2.3.1 Q-Function

The Q-function estimates how good a particular action is when taken in a particular state.

2.4 Exploration vs Exploitation

Exploration means trying new actions to discover better behavior. For example, the robot may try different joint movements during early training.

Exploitation means using actions that have already produced good rewards.

2.5 Episode and Termination

An episode is one complete training attempt starting from the initial state and ending when a termination condition is reached.

2.6 Discount Factor

The discount factor, represented by γ (gamma), determines how much importance is given to future rewards.

3 Algorithms Studied

Learning progression:

Q-Learning → DQN → DDQN → PPO → TRPO → SAC → FlashSAC

3.1 Q-Learning

Q-Learning is a basic value-based RL algorithm that learns the expected value of taking an action in a particular state. It helped us understand the fundamentals of Q-values, rewards, and policy learning.

Flow:

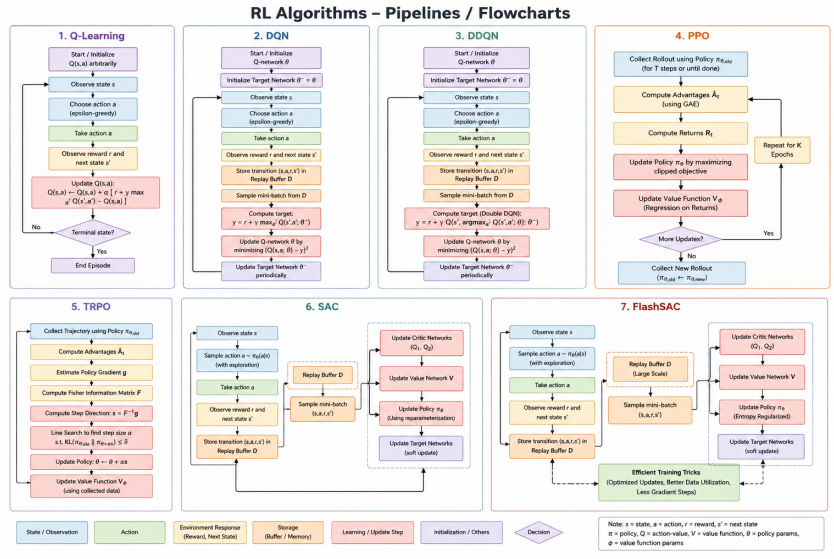

Figure 2: Q-Learning Algorithm Flowchart

3.2 Deep Q-Network (DQN)

DQN extends Q-Learning by replacing the Q-table with a neural network. It introduced important concepts such as Experience Replay and Target Networks, allowing RL to work with larger states.

Flow:

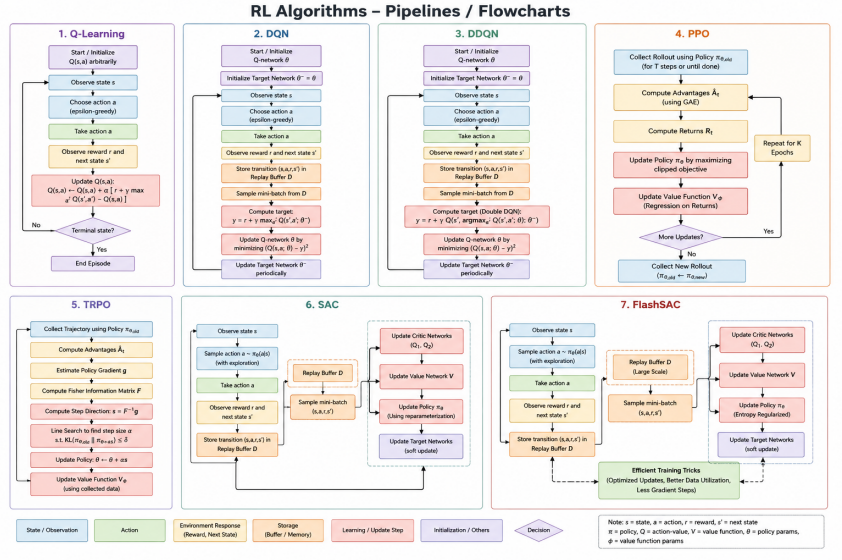

Figure 3: Deep Q-Network (DQN) Algorithm Flowchart

3.3 Double DQN (DDQN)

DDQN improves DQN by reducing the overestimation of Q-values. It separates action selection from action evaluation, resulting in more stable value estimation.

Flow:

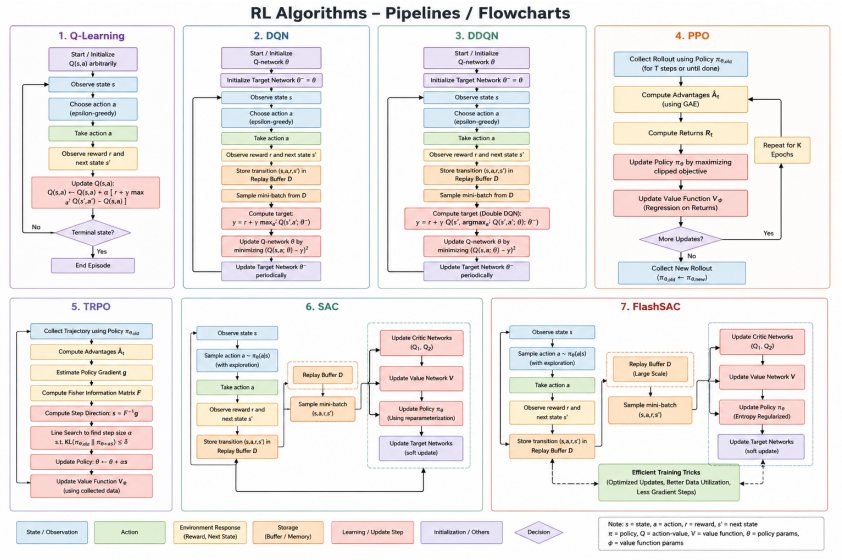

Figure 4: Double DQN (DDQN) Algorithm Flowchart

3.4 Proximal Policy Optimization (PPO)

PPO is a policy-based actor-critic algorithm suitable for continuous control. It uses a clipped objective to prevent large policy updates. PPO became particularly relevant for Waddle because the robot requires continuous joint control.

Flow:

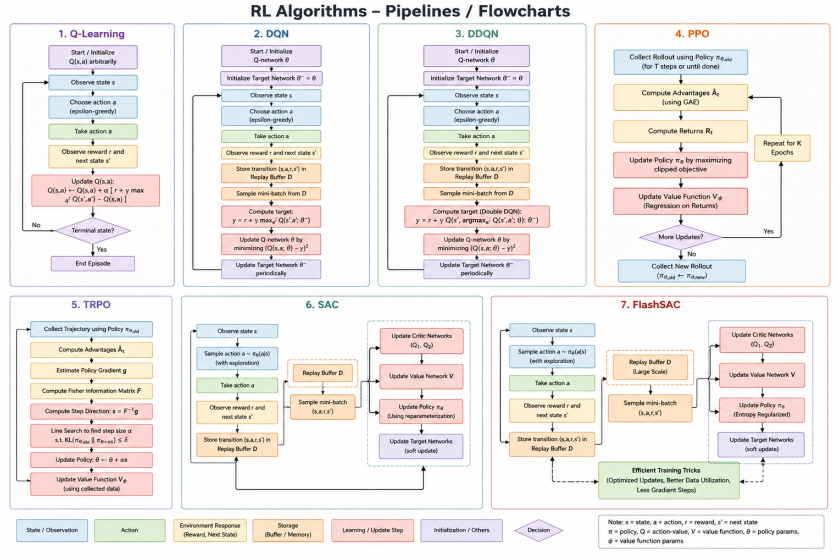

Figure 5: Proximal Policy Optimization (PPO) Flowchart

3.5 Trust Region Policy Optimization (TRPO)

TRPO focuses on stable policy updates by restricting how much the policy can change during training. Studying TRPO helped us understand constrained policy optimization and the motivation behind PPO.

Flow:

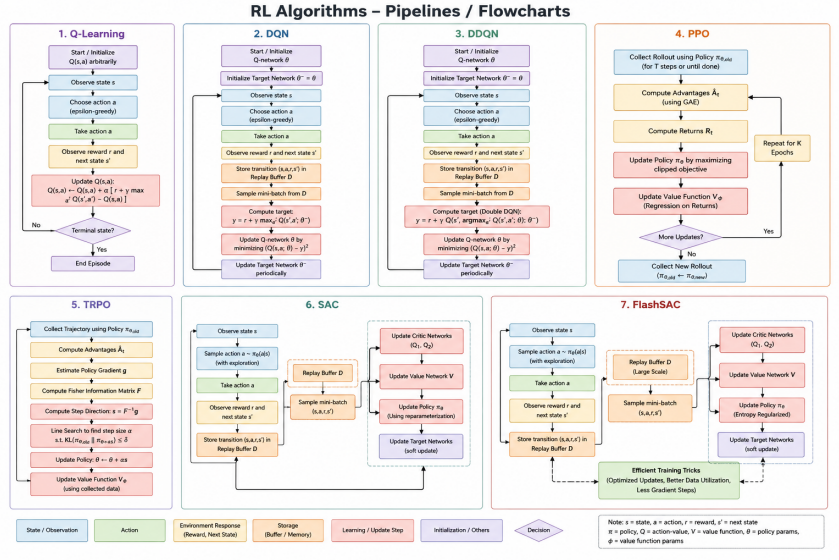

Figure 6: Trust Region Policy Optimization (TRPO) Flowchart

3.6 Soft Actor-Critic (SAC)

SAC is an off-policy actor-critic algorithm designed for continuous action spaces. It uses a replay buffer and entropy regularization to improve exploration and sample efficiency.
Flow:

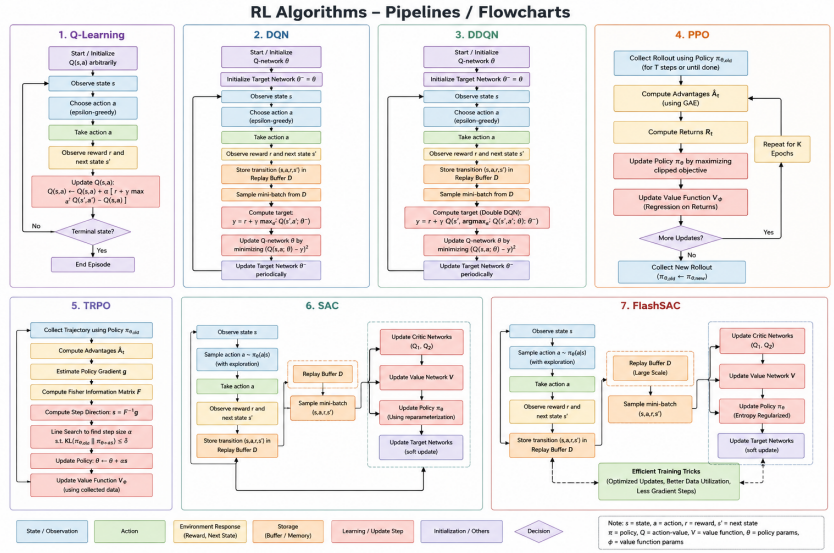

Figure 7: Soft Actor-Critic (SAC) Flowchart

3.7 FlashSAC

FlashSAC builds upon SAC and focuses on improving training efficiency and stability. It introduced us to efficient off-policy training, critic stability, and reducing unnecessary gradient updates.
Flow:

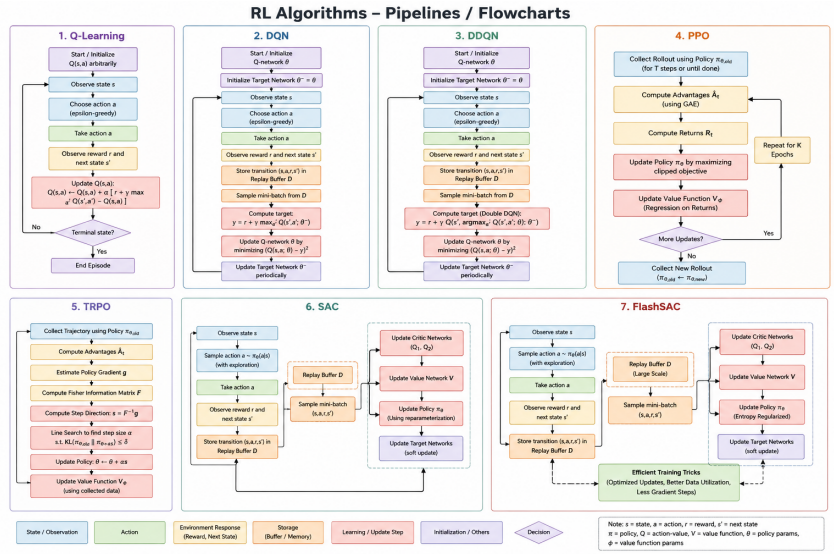

Figure 8: FlashSAC Algorithm Flowchart

4 Simulation Environment

The Waddle project uses a simulated reinforcement learning environment to train and test the robot before deployment on real hardware. Gymnasium is used to create and manage the RL environment, while MuJoCo handles the robot's physics simulation. Instead of directly applying reinforcement learning algorithms to the final robot, we first experimented with different Gymnasium environments. This helped us understand how the nature of the environment, especially its state and action space, affects the choice and performance of an RL algorithm.

4.1 Gymnasium

Gymnasium is a framework used to develop and interact with Reinforcement Learning environments. It provides standard functions such as reset() and step() for communication between the RL algorithm and the environment.

1. Blackjack → (Q-Learning)

We started with the Blackjack environment to understand the basic working of reinforcement learning and Q-Learning. Since the environment has a discrete state and action space, a Q-Table could be used to store the value of each state-action pair.
The agent learned by exploring different actions using an ε-greedy strategy and updating the Q-values based on the received rewards. This environment helped us understand basic concepts before moving to more complex tasks.

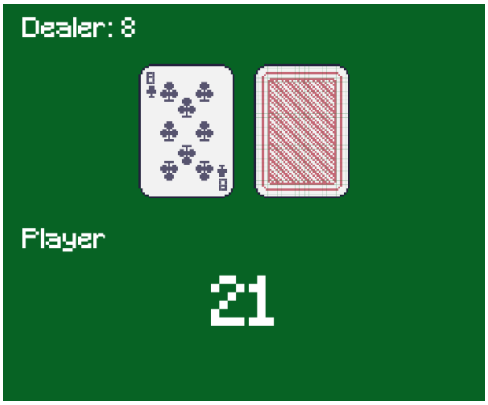

Figure 9: Blackjack

2. Taxi → (Q-Learning)

We then worked with the Taxi environment, where the agent has to navigate through a grid and pick up and drop off a passenger at the correct location. We used Q-Learning because the environment has a discrete action space.
Compared with Blackjack, Taxi provided a more visual understanding of how the learned Q-values affect the agent's movement. The main challenge was balancing exploration and exploitation.

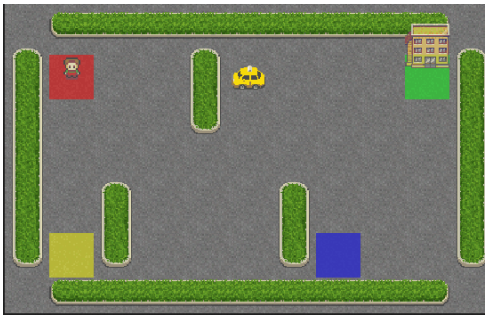

Figure 10: Taxi

3. FrozenLake → (Q-Learning)

FrozenLake was used to further understand Q-Learning in an environment where the agent must reach a goal while avoiding unsafe states. The stochastic nature of the environment showed that the best action is not always guaranteed to produce the expected result. This helped us understand why repeated exploration is required for Q-Learning.

Figure 11: FrozenLake

4. Lunar Lander (DQN and DDQN)

After understanding tabular Q-Learning, we moved to LunarLander, where the problem becomes more complex and a Q-Table becomes less practical. We therefore studied neural-network-based value learning using DQN and DDQN.
DQN replaced the Q-Table with a neural network that estimates Q-values. DDQN was then studied to reduce the overestimation problem in DQN by separating action selection from action evaluation.

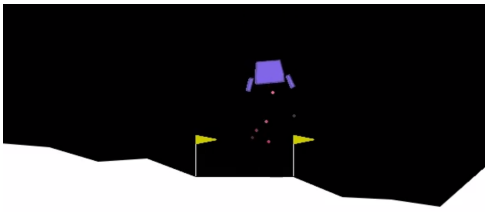

Figure 12: Lunar Lander

5. Bipedal Walker (DQN and DDQN)

We next experimented with BipedalWalker, which is significantly more complex because the robot has to coordinate multiple joints to achieve locomotion.
The original environment uses continuous actions, so action discretization was required for our DQN/DDQN implementation. This experiment showed us the limitations of applying discrete-action algorithms directly to continuous robotic control.

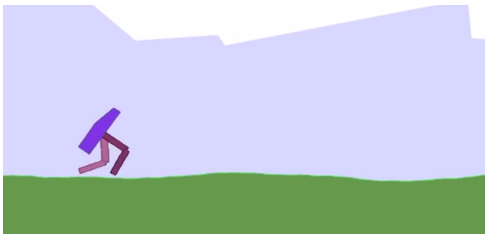

Figure 13: Bipedal Walker

6. Pusher — PPO

We then moved to continuous-control environments and used Pusher with PPO. Unlike DQN/DDQN, PPO can directly handle continuous actions, making it more suitable for robotic control.
The agent learned to control the robot's joints through continuous actions and optimize the predefined reward function. This experiment highlighted the difference between value-based and policy-based approaches.

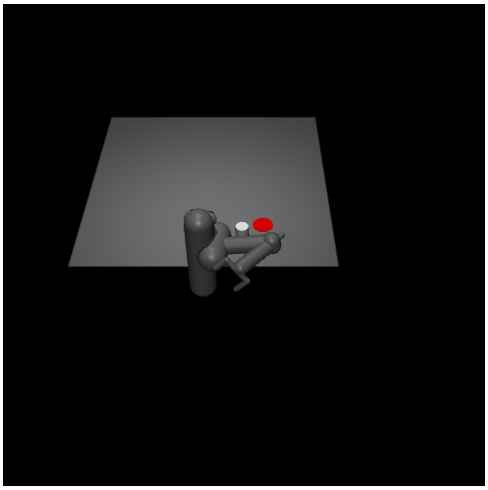

Figure 14: Pusher

7. Walker2D → (PPO and TRPO)

Walker2D was used to study locomotion using PPO and TRPO. The environment already provides a locomotion task and a predefined reward structure.
PPO provided a stable and practical approach for training the walking policy. TRPO was then studied to understand constrained policy updates and how limiting the change between policies can improve training stability.

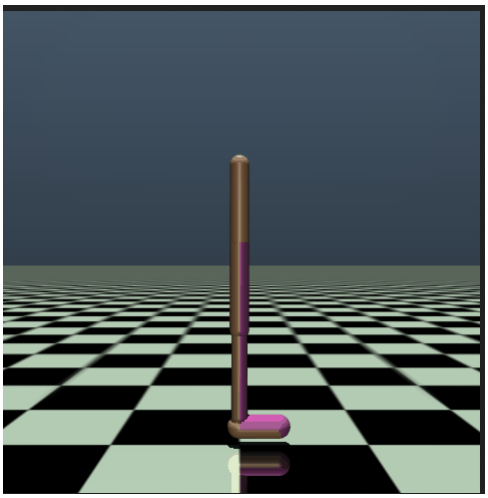

Figure 15: Walker2D

8. Humanoid → (PPO and TRPO)

Humanoid introduced a much more complex locomotion problem with a larger number of joints and a higher-dimensional observation and action space.
This experiment demonstrated that an algorithm that works on simpler environments does not automatically produce good behaviour on a more complex robot. Stable training depends heavily on the reward structure.

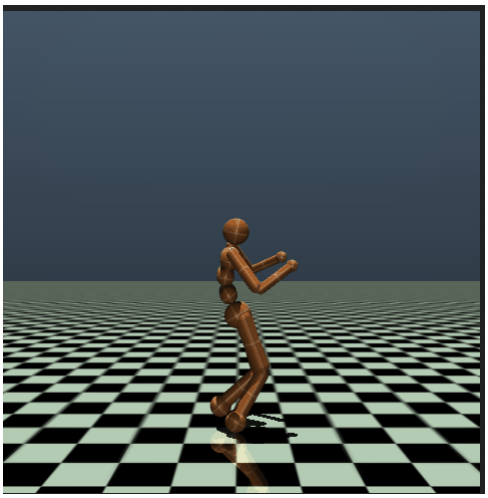

Figure 16: Humanoid

9. HalfCheetah — PPO

HalfCheetah was used with PPO to further study continuous locomotion. The environment focuses on learning coordinated joint movements that produce forward motion.
Compared with the more complex Humanoid environment, HalfCheetah provided a simpler locomotion setup while still requiring continuous control.

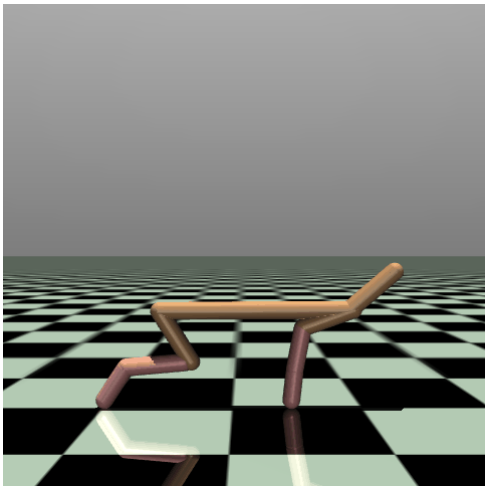

Figure 17: HalfCheetah

5 Environment Structure

The environment contains the robot model, observations, actions, rewards, and termination conditions. The RL algorithm interacts with the environment at every simulation step.

5.1 Observation Space

The observation space defines the information given to the agent. For Waddle, it includes relevant robot information such as joint positions, joint velocities, orientation, and motion-related values.

5.2 Action Space

The action space defines the commands generated by the policy. In Waddle, continuous actions are used to control the robot's joints and produce locomotion.

5.3 Reward Function

The reward function evaluates the robot's behavior after each action. Rewards are designed to encourage desired behavior such as forward movement, velocity tracking, stable posture, and proper walking.

5.4 Episode Termination

An episode ends when a defined termination condition is reached. For example, the robot may fall, become unstable, or reach the maximum episode duration.

6 Reward Engineering

Reward engineering was an important part of Waddle because the quality of the reward directly affected the behavior learned by the robot.

6.1 Reward Weighting

Each reward term is multiplied by a weight to control its importance. The weights were adjusted during experiments to prevent one reward from dominating the others.

6.2 Individual Reward Testing

We tested reward terms individually by keeping one reward active while disabling the others. This helped us understand the effect of each reward on the robot's behavior.

6.3 Reward Ablation and Attempts

Multiple training attempts were performed by changing and testing different reward combinations. The resulting behavior and reward values were checked after each experiment.

6.4 Problems Discovered

One major issue was that the robot could obtain a high total reward without actually walking. The alive and gait rewards were contributing more than the forward-progress reward. This showed that a high numerical reward did not always represent good locomotion.

6.5 Reward Tuning

Based on the experiments, reward weights and reward terms were adjusted. Duplicate or excessive gait rewards were reduced, forward progress was given appropriate importance, symmetry was corrected, and tilt/contact termination conditions were added to encourage stable and meaningful walking behavior.

7 Reward Table

Table 1: Detailed Breakdown of Reward Definitions and Corresponding Weights

Reward/Penalty	**Purpose**	**Effect on Robot**	**Weight**
Linear Velocity Tracking	Match the commanded linear velocity	Moves at the desired speed and direction	2.0
Angular Velocity Tracking	Follow the commanded turning velocity	Turns at the desired rate	0.5
Forward Progress	Encourage forward movement	Encourages actual forward walking	8.0
Heading Drift	Maintain the desired heading	Keeps the robot moving in the correct direction	-1.0
Lateral Path Deviation	Maintain the desired walking path and penalizes it	Keeps the robot close to the desired path	-4.0
Yaw Penalty	Reduce unwanted yaw rotation	Reduces unnecessary turning	-1.0
Gait Phase Tracking	Match foot movement with gait phase	Produces a consistent walking rhythm	1.0
Feet Air Time	Reward appropriate foot lifting	Encourages proper foot lifting and stepping	2.0
Symmetry	Maintain left-right leg coordination	Produces balanced left-right leg movement	-0.3
Flat Orientation	Keep the robot upright	Keeps the robot upright and stable	-2.5
Base Height	Maintain desired body height	Maintains a consistent body height	-1.0
Vertical Velocity	Reduce excessive bouncing and penalize it	Reduces unnecessary bouncing	-2.0
Angular Velocity Penalty	Penalizes excessive body rotation	Reduces excessive body rotation	-0.05
Pelvis Velocity Tracking	Match desired pelvis movement	Keeps the main body moving with the desired velocity	-1.0
Lateral Foot Spread	Maintain stable foot placement	Prevents excessively wide foot placement	-3.0

Table 1: Detailed Breakdown of Reward Definitions and Corresponding Weights (Continued)

Reward/Penalty	**Purpose**	**Effect on Robot**	**Weight**
Gait Phase Contact	Improve footground contact timing	Improves stance and swing timing	1.0
Joint Position Limit	Prevent joint-limit violations	Keeps joints within safe limits	-1.0
Termination Penalty	Penalizes failure/termination	Discourages falling and unstable states	-25.0
Joint Velocity	Reduce excessive velocity	Produces controlled joint movements	-0.0005
Joint Acceleration	Reduce sudden acceleration	Reduces jerky joint motion	-2e-7
Action Rate	Penalizes sudden action changes	Produces smoother control	-0.03
Action Smoothness	Produce smoother actions	Reduces jerky locomotion	-0.015
Alive Reward	Reward staying alive	Encourages the robot to remain upright	1.0

8 PPO Training

PPO was used to train the Waddle robot for continuous locomotion. The training was performed in the simulated environment using the designed observations, actions, and reward function.

8.1 Training Procedure

The policy interacted with the environments and collected rollouts. The collected data was then used to calculate advantages and update the actor and critic networks. This process was repeated for multiple iterations.

$$Environment \rightarrow CollectRollouts \rightarrow CalculateAdvantages \rightarrow PPOUpdate \rightarrow Repeat$$

8.2 Checkpoints

Model checkpoints were saved during training so that different stages of training could be evaluated and the best-performing model could be retained.

8.3 Initial Results

The initial experiments showed that the robot was able to achieve high reward values. However, the reward did not always correspond to meaningful walking behavior.

8.4 Problems Encountered

The major issue was that the robot could obtain a high reward while remaining almost stationary or showing undesirable movement.

8.5 High Reward Without Actual Walking

Further analysis showed that rewards such as alive and gait rewards could contribute significantly to the total reward even when forward movement was poor.

8.6 Diagnosis of Reward Scaling

This indicated a reward-scaling and reward-balancing problem. The individual reward terms were therefore tested separately and their weights were adjusted to make forward locomotion.

9 FlashSAC

FlashSAC was studied after PPO to explore a more efficient off-policy approach based on Soft Actor-Critic.

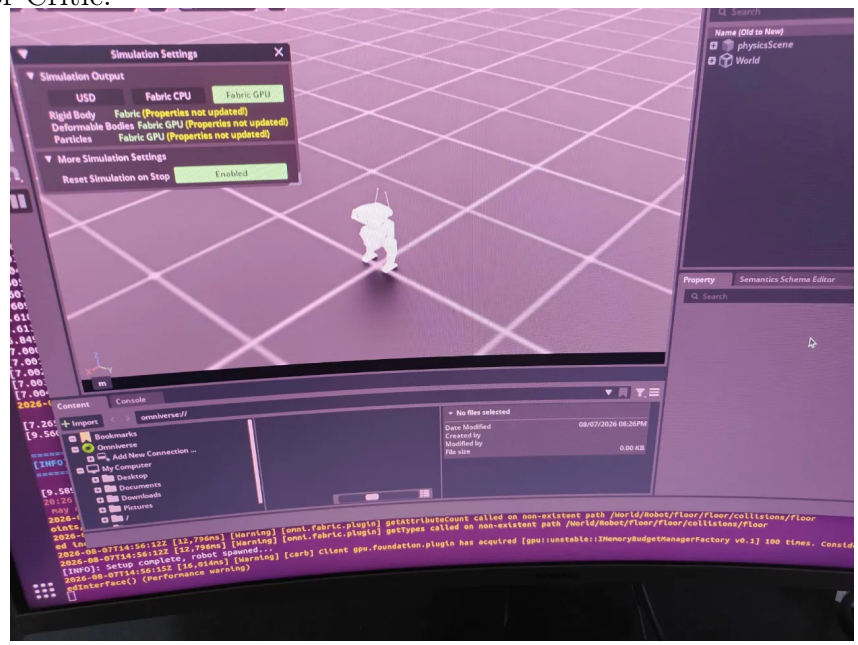

Figure 18: FlashSAC System Architecture

9.1 SAC Fundamentals

SAC is an off-policy actor-critic algorithm designed for continuous control. It uses an actor, critic networks, replay buffer, and entropy-based exploration.

9.2 Off-Policy Learning

In off-policy learning, experiences collected using previous policies can be reused for training. This allows the algorithm to learn from stored past experiences.

9.3 Replay Buffer

The replay buffer stores experiences in the form:

$$(State, Action, Reward, NextState)$$

Samples are later taken from the buffer to train the networks.

9.4 Critic

The critic estimates the Q-value of state-action pairs. It helps determine how good the selected actions are.

9.5 Actor

The actor represents the policy and generates actions based on the current observation.

9.6 Entropy

SAC uses entropy to encourage exploration. This prevents the policy from becoming too deterministic too early and helps it explore different actions.

9.7 Target Networks

Target networks provide more stable targets for critic learning and help reduce instability during training.

10 PPO vs FlashSAC (Comparison)

Table 2: Comparison Between PPO and FlashSAC Algorithms

Aspect	**PPO**	**FlashSAC**
Learning Type	On-policy	Off-policy
Main Approach	Policy optimization	Actor-Critic with Soft Q-Learning
Experience	Uses newly collected rollouts	Reuses past experiences
Replay Buffer	Not used	Used
Exploration	Mainly through policy distribution	Encouraged using entropy
Critic	Estimates value function	Estimates Q-value
Sample Reuse	Limited	High
Training Stability	Generally stable	Designed to improve efficiency while retaining SAC-style stability
Continuous Control	Suitable	Highly suitable
Computational Efficiency	Requires repeated environment rollouts	Can reuse stored experiences
Waddle Application	Initially used for Open Duck Mini	Studied as an alternative after PPO

Why PPO was used first

PPO was selected initially because it is well suited for continuous-control problems and provides a relatively stable way of optimizing the locomotion policy. The Open Duck Mini policy was trained using PPO and its behaviour was evaluated through the reward and walking performance.

However, during PPO training, we observed that high reward did not always correspond to actual walking. This led us to investigate the reward terms and tune the reward structure before considering alternative algorithms.

Why FlashSAC was studied

After working with PPO, we studied SAC and then FlashSAC to understand an off-policy alternative. Unlike PPO, FlashSAC can reuse previously collected experiences through a replay buffer, which makes it attractive for data-efficient continuous-control learning.

FlashSAC was therefore considered for Waddle as an alternative approach for learning locomotion, particularly because robotic simulation can generate large amounts of experience and replaying previous experiences can make better use of that data.

11 Reward Testing and Tuning

Table 3: Observations from Reward Testing and Tuning Iterations

Reward Tested	**Behaviour Observed**	**Learning**
Forward Progress	Robot tried to move forward but could become unstable	Forward movement alone was not sufficient for stable walking
Alive Reward	Robot learned to remain standing but showed little movement	High alive reward could encourage standing instead of walking
Gait Reward	Robot produced leg movements but did not necessarily move forward	Gait motion alone does not guarantee locomotion
Symmetry Reward	Robot produced more coordinated left-right movements	Helped balance the movement of both legs
Velocity Tracking	Robot attempted to match the commanded velocity	Improved directional and speed control
Feet Air-Time	Robot lifted its feet more during movement	Encouraged proper swing phase
Orientation Reward	Robot tried to maintain an upright posture	Reduced falling and excessive tilting
Contact Reward	Foot-ground interaction became more structured	Helped improve stance and stepping behaviour
Combined Rewards	Robot started showing more meaningful locomotion	Multiple complementary rewards were required
Tuned Rewards	Walking became more stable and consistent	Proper reward weighting was critical

12 Conclusion

This project provided a practical understanding of Reinforcement Learning and its application to robotic control. We explored fundamental concepts such as states, actions, rewards, policy, neural networks, and hyperparameter tuning, along with algorithms including DQN, Double DQN, PPO, TRPO, SAC, and FlashSAC. We also implemented and experimented with these approaches using tools such as PyTorch, Gymnasium, and Isaac Lab.

Through this work, we gained valuable experience in designing RL environments, developing reward functions, training agents, and analyzing their performance. Our experiments with FlashSAC helped us understand the potential of efficient off-policy learning for robotic tasks compared to PPO and why it is better. Overall, the project strengthened our foundation in Reinforcement Learning and provided a strong platform for exploring more advanced algorithms and intelligent robotic control systems in the future.

Ultimately, the project can be developed toward real-world autonomous robotic systems capable of learning, adapting, and performing complex tasks with minimal human intervention, contributing to the broader field of intelligent and autonomous robotics.

13 Future Prospects

The future scope of this project lies in extending the current Reinforcement Learning work toward more complex and challenging robotic applications. Future work can also focus on applying these learned policies to complex tasks such as locomotion, navigation, manipulation, and obstacle avoidance. Another important direction is the transition from simulation-based training to deployment on real robotic platforms, which would allow the performance of the learned policies to be evaluated under real-world uncertainties and hardware constraints. These extensions can help develop more efficient, robust, and adaptable learning-based control systems for real-world autonomous robotics.

Ultimately, the project can be developed toward real-world autonomous robotic systems capable of learning, adapting, and performing complex tasks with minimal human intervention, contributing to the broader field of intelligent and autonomous robotics.

14 References

1. Playlist to understand the Reinforcement Learning Fundamentals:
   - https://www.youtube.com/playlist?list=PLqYmG7hTraZDM-OYHWgPebj2MfCFzFObQ
   - https://www.youtube.com/playlist?list=PLwRJQ4m4UJjNymuBM9RdmB3Z9N5-0IlY0
2. Deep Learning and PyTorch
   - Udacity – Deep Learning with PyTorch: https://www.udacity.com/course/deep-learning-pytorch--ud188
   - PyTorch Documentation: https://docs.pytorch.org/docs/2.12/index.html
3. Neural Networks and Hyperparameter Optimization
   - Stanford CS230 – Convolutional Neural Networks: https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-convolutional-neural-networks/
   - IBM – Hyperparameter Tuning: https://www.ibm.com/think/topics/hyperparameter-t 523796994
4. Deep Reinforcement Learning Algorithms:
   - DDQN: https://medium.com/@reetkohli007/double-dqn-a-beginners-guide-to-the-
5. Reinforcement Learning Environments
   - Gymnasium – Blackjack Environment: https://gymnasium.farama.org/environments/toy_text/blackjack/
   - CleanRL: https://github.com/vwxyzjn/cleanrl, https://docs.cleanrl.dev
6. Robotics and Simulation
   - Open Duck Playground: https://github.com/apirrone/Open_Duck_Playground
   - Open Duck Reference Motion Generator: https://github.com/apirrone/Open_Duck_reference_motion_generator
   - Open Duck Blender: https://github.com/pollen-robotics/Open_Duck_Blender
   - Isaac Lab – Reward Functions: https://isaac-sim.github.io/IsaacLab/main/source/api/lab/isaaclab.envs.mdp.html#module-isaaclab.envs.mdp.rewards
7. Research papers
   - https://arxiv.org/pdf/2005.12729
   - https://arxiv.org/pdf/2006.05990
   - https://arxiv.org/pdf/2111.08819
   - https://arxiv.org/pdf/2204.04157